ЧАВО по матрицам и кватернионам

http://www.gamedev.ru/code/articles/faq_matrix_quat

Автор: **Тимур Ж**

Часто задаваемые вопросы по матрицам и кватернионам.

Введение

Замечание по поводу OpenGL и этого документа

В этом документе, как в большинстве книг по математике, все матрицы представлены в стандартном математическом виде. К сожалению, в уроках и документах по использованию графических библиотек типа IrisGL, OpenGL используют представление, в котором изменены местами столбцы и строки.

В этом документе, к примеру, матрица переноса 4x4 записывается в таком виде:

$$M = \begin{pmatrix} 1 & 0 & 0 & X \\ 0 & 1 & 0 & Y \\ 0 & 0 & 1 & Z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

В коде это можно записать вот так:

```
M[0][1] = M[0][2] = M[0][3] =
M[1][0] = M[1][2] = M[1][3] =
M[2][0] = M[2][1] = M[2][3] = 0 ;
M[0][0] = M[1][1] = M[2][2] = m[3][3] = 1 ;
M[3][0] = X ;
M[3][1] = Y ;
M[3][2] = Z ;
```

иначе говоря:

$$M = \begin{pmatrix} M_{00} & M_{10} & M_{20} & M_{30} \\ M_{01} & M_{11} & M_{21} & M_{31} \\ M_{02} & M_{12} & M_{22} & M_{32} \\ M_{03} & M_{13} & M_{23} & M_{33} \end{pmatrix}$$

OpenGL использует одномерный массив для хранения матриц, но, к счастью, они находятся в памяти в таком виде, что получив адрес pfMatrix и скастовав его к float* можно увидеть матрицу в том виде, в каком ее передают в glLoadMatrixf.

Во фрагментах кода в этом документе используются одномерные массивы для хранения матриц. Порядок элементов в них транспонирован как в OpenGL.

Этот документ	OpenGL

$M = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{pmatrix}$	$M = \begin{pmatrix} 0 & 4 & 8 & 12 \\ 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \end{pmatrix}$

Что такое матрица?

Матрица это двумерный массив чисел, где каждый ряд или колонка состоит из одного и более чисел.

Матрицы можно складывать, вычитать, умножать и делить.

Размер матрицы определяется количеством рядов и колонок.

Матрица с M рядов и N колонок описывается как матрица MxN.

Описать отдельный элемент матрицы можно в виде двух индексов.

Используя математическую нотацию индексы обозначают переменными i и j. Сначала пишут строку, затем колонку.

К примеру, если есть матрица M с порядком 4x4, то элементы этой матрицы описываются парами индексов строк и колонок:

$$M = \begin{pmatrix} 00 & 10 & 20 & 30 \\ 01 & 11 & 21 & 31 \\ 02 & 12 & 22 & 32 \\ 03 & 13 & 23 & 33 \end{pmatrix}$$

У верхнего правого элемента матрицы i=0 и j=3, что можно описать так:
$M_{ij} = M_{03}$

В компьютерной анимации обычно используют матрицы 2x2, 3x3 и 4x4.

Матрицы 2x2 используются для двумерного пространства, например, для поворотов, сдвигов и других типов обработки изображений.

Матрицы 3x3 достаточны для хранения информации о повороте (вращении) и масшатбировании. Такие матрицы могут быть использованы для скелетной анимации.

Матрицы 4x3 достаточно, чтобы хранить поворот, масштабирование и перемещение.

Матрицы 4x4 используются для всего набора трансформаций в трёхмерном пространстве, включая перспективное проецирование.

Что такое порядок матрицы?

Порядок матрицы — это её размерность. Матрица из M строк и N столбцов имеет порядок MxN.

Как я могу сделать матрицу в C/C++?

Проще всего использовать ключевое слово typedef.

Матрицы 3x3 и 4x4 могут быть описаны так:

```
typedef float MATRIX3[9];
typedef float MATRIX4[16];
```

Так как матрицы имеют размерность 3x3 и 4x4, им необходимо 9 и 16 элементов соответственно.

На первый взгляд, использование одного линейного массива чисел может показаться не интуитивным. Использование двумерного массива кажется логичнее:

```
typedef float MATRIX3[3][3];
typedef float MATRIX4[4][4];
```

Однако, использование двух систем отсчета для каждого элемента матрицы часто ведет к путанице. В математике, сначала записывают строку(i), а затем колонку(j): M_{ij}

В C/C++, это будет так:

```
matrix[j][i]
```

Использование двумерных массивов также подействует на производительность процессора, так как компилятор C всегда использует операцию умножения для определения индекса элемента.

Так что, эффективнее использовать одномерные массивы. Однако останется решить ещё один вопрос. Как двумерный массив проецируется в линейный?

Есть только два решения: строка сначала, колонка потом, либо колонка сначала, а строка потом.

Разница в производительности минимальна. Если все циклы раскрыты, то нет практически отличий в производительности для таких операций как умножение матрицы на матрицу.

При использовании C/C++ порядок элементов в матрице следующий:

mat[0] = M_{00}, mat[3] = M_{03}, mat[12] = M_{30}, mat[15] = M_{33}.

$$M = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{pmatrix} M = \begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{pmatrix}$$

В чем плюсы от использования матриц?

Один из первых вопросов, задаваемых по использованию матриц в компьютерной анимации: «Зачем вообще они нужны?». Интуиция подсказывает, что придется хорошо потратиться на

циклы, и тем самым матричные перемножения замедлят программу.

Решением этой проблемы, являются регистры процессора и процессорный кэш. Первый может быть счетчиком в цикле, а алгоритм предсказания с заполнением кэша позволит оптимизировать доступ к памяти.

Также можно указать и на плюсы. Используя этот математический подход в описании 3D алгоритмов, можно предсказать и спланировать систему 3D анимации. Этот подход позволяет реализовать анимацию персонажа, сплайны и инверсную кинематику.

Но чаще всего звучит такой вопрос: «А не будет ли быстрее, просто умножить каждую пару координат на коэффициенты поворота для оси, вместо того чтобы производить полное векторно-матричное умножение?»

иначе говоря:
Вращение в X преобразует Y и Z
Вращение в Y преобразует X и Z
Вращение в Z преобразует X и Y

За это приводятся следующие аргументы:

Дана вершина V = (x, y, z), углы поворота (A,B и C) и перенос (D,E,F).

И следующий алгоритм:

```
sx = sin(A)             // Настройка - делается единожды
cx = cos(A)
sy = sin(B)
cy = cos(B)
sz = sin(C)
cz = cos(C)

x1 =  x * cz +  y * sz  // Поворот каждого вертекса
y1 =  y * cz -  x * sz
z1 =  z

x2 = x1 * cy + z1 * sy
y2 = z1
z2 = z1 * cy - x1 * sy

x3 = x2
y3 = y2 * cx + z1 * sx
z3 = z2 * cx - x1 * sx

xr = x3 + D             // Перенос каждого вертекса
yr = y3 + E
zr = z3 + F
```

Вместе они займут следующее количество процессорного времени:

Настройка	На одну вершину
6 тригонометрических функций 6 присваиваний.	12 присваиваний 12 умножений 9 сложений

Те же самые операции, но при использовании матричного умножения.

С матрицей 4x4 они займут:

Настройка	Изменение	На одну вершину	Изменение
6 тригонометрических функций	0		0
18 присваиваний	-12	3 присваиваний	-9
12 умножений	+12	9 умножений	-3
6 вычитаний	+6	6 сложений	-3

Сравнивая две таблицы, видно, что матрица поворота стоит как минимум 12 умножений и дополнительно 18 присваиваний.

Однако мы экономим на обработке вершин. Использование матричного умножения, от настройки всего 4 вершин, перевесит дополнительные затраты на настройке.

Как матрицы влияют на систему координат?

Матрицы поворота, переноса, сдвига очень просто действуют на систему координат.

Первые три колонки матрицы описывают направление осей X,Y,Z соответственно.

Если описана матрица 4x4 как:

$$M = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{pmatrix}$$

Вектор направления для каждой оси будет следующий:

ось $X = [AEI]$
ось $Y = [BFJ]$
ось $Z = [CGK]$

АРИФМЕТИКА

Что такое единичная матрица?

Единичная матрица — матрица, имеющая равное число строк и столбцов. И все элементы с индексами i=j равны 1, остальные равны 0.

К примеру, единичная матрица 4x4:

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Что такое главная диагональ матрицы?

Главная диагональ матрицы - это набор элементов, у которых номер ряда равен номеру колонки, иначе говоря:

$M_{i,j}$, где $i = j$

Так что у единичной матрицы, лишь элементы главной диагонали равны 1, а все остальные 0.

Что такое транспонирование матрицы?

Транспонированная матрица это матрица, в которой поменяли все элементы матрицы относительно главной диагонали.

Это можно выразить математической операцией вида:
$M'_{i,j} = M_{j,i}$

Однако такое можно сделать лишь с матрицей с равным числом строк и столбцов.

Если матрица M равна:
$$M = \begin{pmatrix} 0.707 & -0.866 \\ 0.866 & 0.707 \end{pmatrix}$$

Транспонированная равна:
$$M = \begin{pmatrix} 0.707 & 0.866 \\ -0.866 & 0.707 \end{pmatrix}$$

Если имеется матрица поворота, то транспонирование гарантирует, что у матрицы есть обратная.

Как мне сложить две матрицы?

Правило сложения двух матриц: «добавь строку и колонку к строке и колонке».

Математически это выглядит так: $R_{i,j} = M_{i,j} + L_{i,j}$

При этом матрицы должны быть одного размера.

К примеру, если матрица 3x3 M складывается с матрицей 3x3 L результат будет таким:
$$R = M + L = \begin{pmatrix} A & B & C \\ D & E & F \\ G & H & I \end{pmatrix} + \begin{pmatrix} J & K & L \\ M & N & O \\ P & Q & R \end{pmatrix} = \begin{pmatrix} A+J & B+K & C+L \\ D+M & E+N & F+O \\ G+P & H+Q & I+R \end{pmatrix}.$$

Как мне вычесть две матрицы?

Правило вычитания двух матриц: «вычитай строку и колонку из строки и колонки».

Математически будет так: $R_{i,j} = M_{i,j} - L_{i,j}$

Матрицы должны быть строго одного размера.

К примеру, если матрица 3x3 L вычитается из матрицы 3x3 M результат будет таким:
$$R = M - L = \begin{pmatrix} A & B & C \\ D & E & F \\ G & H & I \end{pmatrix} - \begin{pmatrix} J & K & L \\ M & N & O \\ P & Q & R \end{pmatrix} = \begin{pmatrix} A-J & B-K & C-L \\ D-M & E-N & F-O \\ G-P & H-Q & I-R \end{pmatrix}.$$

Как мне умножить две матрицы?

Правило такое: «умножай строку на колонку и сумма будет результатом.»

Математически: $R_{i,j} = \sum_{i=1}^{n} M_{i,j} L_{j,i}$

Если есть две матрицы:
M_{AB} и L_{CD}

то величины B и C должны быть равны.

Результат будет R_{AD}

Так что, можно умножить матрицу 4xN с матрицей 4x4, но не наоборот.

К примеру, если матрица 4x4 M описана так:

$$M = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{pmatrix}$$

и матрица 4x2 L описана так:

$$L = \begin{pmatrix} Q & R \\ S & T \\ U & V \\ W & X \end{pmatrix}$$

то размер результата будет 2x4:

$$R = M * L = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{pmatrix} * \begin{pmatrix} Q & R \\ S & T \\ U & V \\ W & X \end{pmatrix}$$

$$= \begin{pmatrix} AQ+BS+CU+DW & AR+BT+CV+DX \\ EQ+FS+GU+HW & ER+FT+GV+HX \\ IQ+JS+KU+LW & IR+JT+KV+LX \\ MQ+NS+OU+PW & MR+NT+OV+PX \end{pmatrix}$$

Как мне взять корень или возвести матрицу в квадрат?

Из матрицы можно взять корень или возвести в целую степень. Однако есть несколько ограничений. Для любого возведения матрица должна быть квадратная, иначе говоря, количество столбцов должо быть равно количеству строк.

К примеру,
M^{-1} — обратная матрица,

M^{0} — единичная матрица,

M^{1} — матрица не меняется,

M^{2} — возводит в квадрат,

M^{3} — куб матрицы.

Возведение матрицы в степень больше одного означает умножить матрицу на себя n-число раз.

К примеру:

$$M^2 = M \times M$$

$$M^3 = M \times M \times M$$

и так далее.

Возведение в степень единичной матрицы всегда даёт единичную матрицу: $I^n = I$

Как мне умножить один и более векторов на матрицу?

Лучший способ выполнить это задание представить список векторов в виде одной матрицы, где каждый вектор представлен одной колонкой.

Если N векторов должны быть умножены на матрицу 4x4, они могут быть выражены в виде одной матрицы 4xN:

$$M = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{pmatrix}$$

а список векторов:

$$V = \begin{pmatrix} x_1 & x_2 & x_3 & x_4 & x_5 \\ y_1 & y_2 & y_3 & y_4 & y_5 \\ z_1 & z_2 & z_3 & z_4 & z_5 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Заметьте, что вся дополнительная четвертая строка равна константе 1. Она не играет никакой роли, используется, чтобы порядок матрицы M совпадал с порядком списка V.

Умножение производится так:

$$M*V = V'$$

$$\begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{pmatrix} * \begin{pmatrix} x_1 & x_2 & x_3 & x_4 & x_5 \\ y_1 & y_2 & y_3 & y_4 & y_5 \\ z_1 & z_2 & z_3 & z_4 & z_5 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} A*x_1+B*y_1+C*z_1+D & A*x_2+B*y_2+C*z_2+D & \dots \\ E*x_1+F*y_1+G*z_1+H & E*x_2+F*y_2+G*z_2+H & \dots \\ I*x_1+J*y_1+K*z_1+L & I*x_2+J*y_2+K*z_2+L & \dots \\ M*x_1+N*y_1+O*z_1+P & M*x_2+N*y_2+O*z_2+P & \dots \end{pmatrix}$$

На каждый вектор в списке приходится в общем 12 умножений, 16 сложений и 1 деление для перспективы.

Если известно, что это матрица поворота или переноса, то можно не делить.

Определитель и обратная матрица

Что такое определитель матрицы?

Определитель матрицы это число, которое показывает, есть ли у матрицы обратная или нет. Если определитель равен 0, то обратной матрицы нет, иначе обратная матрица существует.

К примеру, есть матрица из одного элемента: $M = (m_{0,0})$. Для матрицы такого размера, определитель это просто значение одного элемента.

А, обратная матрица: $M^{-1} = \left(\frac{1}{m_{0,0}}\right)$.

Если элемент, а соответственно и определитель, не равен 0, то обратная матрица существует.

В случае единичной матрицы ее значение будет 1/1 или 1.0

Однако, если это число 0, то и определитель тоже 0.

При попытке найти обратное от нуля, получаем бесконечность. Так делать нельзя, так что для 0 не существует обратной матрицы.

Для единичной матрицы определитель всегда равен 1. Все матрицы с определителем равным единице называют изотропными.

Поэтому все матрицы вращения изотропны, так как определитель всегда равен 1.0.

Что можно доказать так:

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix} = \begin{pmatrix} cos(x) & -sin(x) \\ sin(x) & cos(x) \end{pmatrix}$$

$$D = AD - BC$$

$$D = (cos(x) * cos(x)) - (-sin(x) * sin(x))$$

$$D = cos^2(x) + sin^2(x)$$

но мы знаем, что:

$$cos^2(x) + sin^2(x) = 1$$

Так что $D = 1$

Как мне посчитать определитель матрицы?

Определитель матрицы считается при помощи правила Крамера,
где каждое число может быть рассчитано
через разбиение матрицы в более малые матрицы.

Для 2x2 матрицы M, определитель B считается так:

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}$$

$$D = AD - BC$$

Для матриц размером 3x3 и 4x4, найти определитель сложнее,
но все равно можно при помощи методов типа правила Крамера.

Что такое изотропная и анизотропная матрицы?

Изотропной матрицей называют матрицу, в которой сумма квадратов всех трех строк или колонок равна 1.

Матрицы, в которой это не происходит, называется Анизотропной.

Матрицы используются для того, чтобы поворачивать и скалировать объект, иногда может быть необходимо, увеличивать или сжимать по одной оси сильнее, чем по другим.

К примеру, в сейсмических задачах, необходимо увеличить ось Z на 50 или более, при этом оси X и Y нужно оставить неизменными.

Другим примером может служить «раздавливание» и «растягивание» в анимации персонажей. Когда персонаж ударяет тяжелый предмет типа наковальни, то необходим эффект, при котором персонаж растягивается в стороны и сдавливается вертикально:

Матрица будет выглядеть так:

$$M = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Однако здесь появляется проблема. В то время как не будет проблем с обработкой вершин, мы получим неправильное освещение, из-за кривых нормалей.

На шаге обработки данных, трансформация, выраженная данной матрицей Изменит как вершины, так и нормали.

После такого умножения, все нормали перестанут быть нормализованными, что подействует на освещение и обратное отсечение.

Что такое обратная матрица?

Дана матрица M^{-1}, то обратная матрица этой матрицы, записывается как M, это матрица должна удовлетворять условию:

$$M \times M^{-1} = I$$

где I единичная матрица.

Так что, в результате умножения матрицы на обратную к ней будет единичная матрица. Однако матрица должна удовлетворять нескольким условиям, перед тем как обратная матрица будет посчитана.

Во-первых, ширина и высота матрицы должны совпадать, во-вторых ее определитель не должен равняться 0.

Расчет обратной матрицы необходим для реализации обратной кинематики с помощью сплайнов.

Как мне посчитать обратную матрицу?

В зависимости от размера матрицы, расчет обратной может быть простым или очень сложным.

К примеру, обратная 1x1 матрица это просто обратная от одного элемента:

$$M = (x)$$

Обратная матрица:

$$M^{-1} = \left(\frac{1}{x}\right).$$

Решение 2x2 матриц и больших может быть получено при использовании правила Крамера или через решение системы уравнений.

Однако, для конкретных случаев, таких как единичная матрица или матрицы поворота, обратная матрица сразу известна или может быть получена транспонированием.

Как мне посчитать обратную матрицу от единичной матрицы?

Обратная единичной матрицы и есть единичная матрица.

$$I * I^{-1} = I$$

У любой единичной матрицы определитель всегда равен 1.

Как мне посчитать обратную от матрицы вращения?

Так как определитель матрицы вращения равен 1,
то расчет обратной матрицы равен транспонированию.

Альтернативно, если известны углы поворота, то можно взять их с минусом,
для расчета новой матрицы вращения.

Как мне посчитать обратную матрицу при помощи правила Крамера?

Дана 3x3 матрица M:

$$M = \begin{pmatrix} A & B & C \\ D & E & F \\ G & H & I \end{pmatrix}$$

То ее определитель считается так:

$$detM = sum_{i=1}^{n} M * submat_{0,i} M * -1^{i}$$

где:

```
подмат   M - матрица собранная из всех строк и колонок матрицы M
      ij
без строки i и колонки j. подмат    может вызываться рекурсивно.
                                 i,j
```

Если определитель не равен 0, обратная матрица существует.

В этом случае, значение каждого элемента можно определить так:

```
 -1      1                            i+j
M    = -----  *  det подмат     M * -1
 j,i   det M               i,j
```

Как мне посчитать обратную от 2x2 матрицы?

Для матриц 2x2, расчет несколько сложнее. Если матрицы определена так:

```
    | A B |
M = |     |
    | C D |
```

То определитель равен:

```
det = AD - BC
```

И обратная задается так:

```
 -1    1   |  D  -B  |
M  =  ---  |         |
       det | -C   A  |
```

Это можно доказать используя правило Крамера. Дана матрица M:

```
    | A B |
M = |     |
    | C D |
```

Ее определитель равен:

```
                        0                            1
det =  M    * подмат M    * -1 +  M    * подмат M    * -1
        0,0           0,0          0,1           0,1

<=>    M    * M    * 1         +  M    * M    * -1
        0,0    1,1                 0,1    1,0

<=>    A  * D                  +  B    * C    * -1

<=>    AD                      +  BC . -1

<=>    AD - BC
```

==========

И обратная делается так:

```
 -1                      0+0      -1
M    = det submat    * -1    <=> M    = M    *  1 <=> D
 0,0             0,0              0,0    1,1

 -1                      1+0      -1
M    = det submat    * -1    <=> M    = M    * -1 <=> C * -1
 0,1             1,0              0,1    1,0

 -1                      0+1      -1
M    = det submat    * -1    <=> M    = M    * -1 <=> B * -1
 1,0             0,1              1,0    0,1

 -1                      1+1      -1
M    = det submat    * -1    <=> M    = M    *  1 <=> A
 1,1             1,1              1,1    0,0
```

То обратная матрица равна:

```
 -1    1  |  D  -C |
M   = --- |        |
      det | -B   A |
```

Если ее определитель не равен 0.

Как мне посчитать обратную от 3x3 матрицы?

Для матриц 3x3 и больше, обратная считается с помощью правила Крамера или через решение системы линейных уравнений.

Если правило Крамера применить к матрице M:

```
  | A B C |
M = | D E F |
    | G H I |
```

То ее опеделитель считается так:

```
det M = A * (EI - HF) - B * (DI - GF) + C * (DH - GE)
```

Если определитель не равен 0, то обратная считается так:

```
 -1     1     |   EI-FH  -(BI-HC)   BF-EC  |
M   = ----- . | -(DI-FG)   AI-GC  -(AF-DC) |
      det M   |   DH-GE  -(AH-GB)   AE-BD  |
```

Это можно реализовать с помощью пары Сишных функций:

```
 VFLOAT m3_det( MATRIX3 mat )
   {
   VFLOAT det;

   det = mat[0] * ( mat[4]*mat[8] - mat[7]*mat[5] )
       - mat[1] * ( mat[3]*mat[8] - mat[6]*mat[5] )
       + mat[2] * ( mat[3]*mat[7] - mat[6]*mat[4] );

   return( det );
   }

----------------------------------

void m3_inverse( MATRIX3 mr, MATRIX3 ma )
  {
  VFLOAT det = m3_det( ma );

  if ( fabs( det ) < 0.0005 )
    {
    m3_identity( ma );
    return;
    }

  mr[0] =    ma[4]*ma[8] - ma[5]*ma[7]   / det;
  mr[1] = -( ma[1]*ma[8] - ma[7]*ma[2] ) / det;
  mr[2] =    ma[1]*ma[5] - ma[4]*ma[2]   / det;

  mr[3] = -( ma[3]*ma[8] - ma[5]*ma[6] ) / det;
  mr[4] =    ma[0]*ma[8] - ma[6]*ma[2]   / det;
  mr[5] = -( ma[0]*ma[5] - ma[3]*ma[2] ) / det;

  mr[6] =    ma[3]*ma[7] - ma[6]*ma[4]   / det;
  mr[7] = -( ma[0]*ma[7] - ma[6]*ma[1] ) / det;
  mr[8] =    ma[0]*ma[4] - ma[1]*ma[3]   / det;
  }
```

Как мне посчитать обратную 4x4 матрицы?

Также как с матрицами 3x3, нужно применить правило Крамера или решить линейные уравнения.

Эффективно будет использовать уже описанные Сишные функции, которые использовались при расчете обратной к 3x3 матрице.

Чтобы реализовать правило Крамера для матриц 4x4, необходимо выделить отдельные подматрицы. Это достигается при помощи следующей функции:

```
  void m4_submat( MATRIX4 mr, MATRIX3 mb, int i, int j )
   {
   int ti, tj, idst, jdst;

   for ( ti = 0; ti < 4; ti++ )
```

```
    {
    if ( ti < i )
      idst = ti;
    else
      if ( ti > i )
        idst = ti-1;

    for ( tj = 0; tj < 4; tj++ )
      {
      if ( tj < j )
        jdst = tj;
      else
        if ( tj > j )
          jdst = tj-1;

      if ( ti != i && tj != j )
        mb[idst*3 + jdst] = mr[ti*4 + tj ];
      }
    }
  }
```

Определитель матрицы 4x4 может быть расчитан так:

```
VFLOAT m4_det( MATRIX4 mr )
  {
  VFLOAT  det, result = 0, i = 1;
  MATRIX3 msub3;
  int     n;

  for ( n = 0; n < 4; n++, i *= -1 )
    {
    m4_submat( mr, msub3, 0, n );

    det     = m3_det( msub3 );
    result += mr[n] * det * i;
    }

  return( result );
  }
```

Обратная матрица может быть рассчитана так:

```
int m4_inverse( MATRIX4 mr, MATRIX4 ma )
{
VFLOAT  mdet = m4_det( ma );
MATRIX3 mtemp;
int     i, j, sign;

if ( fabs( mdet ) < 0.0005 )
  return( 0 );
```

```
  for ( i = 0; i < 4; i++ )
    for ( j = 0; j < 4; j++ )
      {
      sign = 1 - ( (i +j) % 2 ) * 2;

      m4_submat( ma, mtemp, i, j );

      mr[i+j*4] = ( m3_det( mtemp ) * sign ) / mdet;
      }

  return( 1 );
  }
```

Очень полезно иметь функции, которые могут рассчитать

обратную любой 4x4 матрицы. Их применение включает возможность посчитать основную матрицу для сплайнов, обратить вращение и перестроить матричные уравнения.

Как мне посчитать обратную матрицу с помощью линейных уравнений?

Если есть матрица M:

```
     | A B C |
M  = | D E F |
     | G H I |
```

то есть обратная матрица:

```
     | P Q R |
M' = | S T U |
     | V W X |
```

и верно следующее выражение:

```
                -1
   M     .     M     = I

| A B C |   | P Q R |   | 1 0 0 |
| D E F | . | S T U | = | 0 1 0 |
| G H I |   | V W X |   | 0 0 1 |
```

Обратная может быть рассчитана через решение системы линейных уравнений:

```
| AP + BS + CV |   | 1 |   Колонка 0 (X)
| DP + ES + FV | = | 0 |
| GP + HS + IV |   | 0 |

| AQ + BT + CW |   | 0 |   Колонка 1 (Y)
```

```
| DQ + ET + FW | = | 1 |
| GQ + HT + IW |   | 0 |

| AR + BU + CX |   | 0 |   Колонка 2 (Z)
| DR + EU + FX | = | 0 |
| GR + HU + IX |   | 1 |
```

ТРАНСФОРМАЦИИ

Что такое матрица поворота?

Матрица поворота используется для поворота точек вокруг
системы координат. В то время как отдельные точки привязываются к новым координатам, расстояния между ними не меняются.

Все вращения определяются через тригонометрические функции синус и косинус.

Для двумерной системы координат, матрица вращения такая:
$\begin{pmatrix} \cos(A) & -\sin(A) \\ \sin(A) & \cos(A) \end{pmatrix}$
Если угол A равен 0, то на выходе получаем единичную матрицу:
$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
Если угол A равен +90 градусов, то матрица такова:
$M = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$
Если угол A равен -90 градусов, то матрица такова:
$I = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$

Углы поворота, взятые с обратным знаком равны транспонированию матрицы.

Если матрица вращения умножена на транспонированную,
в результате будет единичная матрица

Как матрицы вращения влияют на систему координат?

Матрицы вращения влияют на систему координат следующим образом:

По договоренности положительные углы создают вращение по часовой стрелке,
если смотреть из центра координат в положительном направлении оси вращения.

Следуя этому правилу, в праворукой декартовой системе координат,
мы увидим следующие три вида:

```
+----------------------------------------+
```

```
|                                        |
|   X-axis       Y-axis        Z-axis    |
|                                        |
|                                        |
|  ^ Y          ^ Z                Y ^   |
|  |            |                    |   |
|  |            |                    |   |
|  |            |                    |   |
|  |            |                    |   |
|  O----> Z     O----> X      X <----O   |
|                                        |
+----------------------------------------+
```

Так как вращение на положительный угол создает вращение по часовой стрелке, можно создать набор координат для каждого такого вращения.
Для упрощения рассмотрим поворот на +90:

Для X-оси:

$(0,1,0)->(0,0,1)$
$(0,0,1)->(0,-1,0)$
$(0,-1,0)->(0,0,-1)$
$(0,0,-1)->(0,1,0)$

Можно упростить так:
$X' = X$
$Y' = -Z$
$Z' = Y$
В матрице это будет так:
$$R_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & cosA & -sinA \\ 0 & sinA & cosA \end{pmatrix}$$
То же самое для Y-оси:
$(0,0,1)->(1,0,0)$
$(1,0,0)->(0,0,-1)$
$(0,0,-1)->(-1,0,0)$
$(-1,0,0)->(0,0,1)$
Упрощаем:
$X' = Z$
$Y' = Y$
$Z' = -X$
В матрице:
$$R_y = \begin{pmatrix} cosA & 0 & sinA \\ 0 & 1 & 0 \\ -sinA & 0 & cosA \end{pmatrix}$$
Z-ось:
$(0,1,0)->(-1,0,0)$
$(-1,0,0)->(0,-1,0)$
$(0,-1,0)->(1,0,0)$
$(1,0,0)->(0,1,0)$
Упрощаем:
$X' = -Y$
$Y' = X$
$Z' = Z$

В матрице:

$$R_z = \begin{pmatrix} cosA & -sinA & 0 \\ sinA & cosA & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Это и есть те три базовые матрицы, которые используются в OpenGL.

Как сгенерировать матрицу поворота по оси X?

Для 4x4 матрицы:

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(A) & -sin(A) & 0 \\ 0 & sin(A) & cos(A) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Как сгенерировать матрицу поворота по оси Y?

Для 4x4 матрицы:

$$M = \begin{pmatrix} cos(A) & 0 & -sin(A) & 0 \\ 0 & 1 & 0 & 0 \\ sin(A) & 0 & cos(A) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Как сгенерировать матрицу поворота по оси Z?

Для 4x4 матрицы:

$$M = \begin{pmatrix} cos(A) & -sin(A) & 0 & 0 \\ sin(A) & cos(A) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Что такое углы Эйлера?

Углы Эйлера это имя, данное для набора углов поворота определённых относительно трех осей X,Y,Z

Их можно определить в векторном виде (x,y,z) и хранить в структуре VECTOR.

К примеру, набор из $(0,0,0)$ всегда дает единичную матрицу.

Если представить углы в таком виде, то:
$(90,0,0)$ вращение на +90 градусов по X.
$(0,90,0)$ вращение на +90 градусов по Y.
$(0,0,90)$ вращение на +90 градусов по Z.

Что такое рыск, качение и тангаж?

Рыск, качение и тангаж это термины аэронавтики для поворота в Эвклидовой системе координат (Углах Эйлера), относительно местной системы координат самолета.

Представьте, что вы в самолете и смотрите вперед

Ось Z соединяет хвост и нос самолета.
X от конца левого крыла к концу правого крыла.
Y указывает с земли в небо.

Тангаж- поворот по X, Рыск - поворот по Y, Качение - поворот по Z.

Как мне комбинировать матрицы поворота?

Матрицы поворота комбинируются при помощи матричного умножения.
Так что порядок умножения очень важен.

Что такое фиксация оси?

Фиксация - проблема, появляющаяся при использовании углов Эйлера.
Так как конечный поворот зависит от порядка умножения, может произойти так, что одна ось вращения отобразиться на другую ось.

Из-за чего станет невозможно повернуть объект в нужной оси.

К примеру, если вращать объект в порядке Z,Y,X и повернуть по оси Y на 90 градусов.

В этом случае, вращение по оси Z будет произведено первым, а значит, будет проведено, верно. По оси Y также корректно. Однако после вращения по Y, ось X будет вращаться по оси Z.

Так что любое вращение по X на самом деле будет вращать по Z.

Единственное решение этой проблемы - использовать Кватернионы.

Как правильно комбинировать матрицы вращения?

На самом деле, нет "правильного пути". Однако для возможности предсказать результат, некоторая организация необходима. Это также нужно, при создании полноценной 3D матричной библиотеки.

Самый простой путь повернуть объект:

$$M = X * Y * Z$$

где M финальная матрица вращения, и X,Y,Z матрицы вращения по осям.
Они опишет поворот по оси X (тангаж), затем по Y(рыск) и в конце по Z(качение).

Однако со стороны наблюдателя порядок вращений будет обратным.

К примеру, если ты стоишь и поворачиваешься налево, то все в поле твоего зрения движется вправо.

Однако кто-либо кто смотрит на тебя, скажет, что ты повернулся направо.

Так что вид из камеры нужно моделировать в таком порядке:

$$M = (-Z)*(-Y)*(-X)$$

Это обратная матрица вращения генерируется,
если камера считается как еще один объект.

Как сгенерировать матрицу вращения из углов Эйлера?

На первый взгляд, самый явный способ создать матрицу вращения из углов Эйлера
это создать каждую матрицу отдельно и умножить их друг на друга.

```
m3_rotx( mat_x,     vec -> angle_x );
m3_roty( mat_y,     vec -> angle_y );
m3_rotz( mat_z,     vec -> angle_z );
m3_mult( mat_tmp,   mat_z, mat_y   );
m3_mult( mat_final, mat_tmp, mat_x );
```

Из них можно сделать отдельную функцию:

```
m3_fromeuler( MATRIX *mat_final, VECTOR3 *euler )
```

Однако этот способ очень затратный в терминах времени обработки.
Для матрицы 4x4 гарантированно 10 элементов будут равны 0, два равны 1,
и четыре оставшихся будут иметь какое-то значение,
больше 75% всех матричных операций пройдет впустую. И это не считая
настройку и инициализацию каждой матрицы.

Вместе, больше 75% всех операций будут потрачены на нулевой или единичный результат.

Нужно более эффективное решение. К счастью, есть другой путь, определения
конечной матрицы.

Если все три матрицы записать в алгебраической форме,
то мы получим следующее выражение:

$$M = X*Y*Z$$

Где M - конечная матрица,
X - матрица вращения по X,
Y - матрица вращения по Y,
Z - матрица вращения по Z,

Расписав мы получим:

$$X = \begin{pmatrix} 1 & 0 & 0 \\ 0 & A & -B \\ 0 & B & A \end{pmatrix}$$

$$X = \begin{pmatrix} C & 0 & -D \\ 0 & 1 & 0 \\ D & 0 & C \end{pmatrix}$$

$$X = \begin{pmatrix} E & -F & 0 \\ F & E & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

где A,B косинус и синус вращения по оси X,
C,D косинус и синус вращения по оси Y,
E,F косинус и синус вращения по оси Z,

То выражение:

```
M  = X.Y.Z
```

Можно разбить на два матричных умножения:

```
M' = X.Y

M  = M'.Z
```

Посчитаем сначала M':

```
M' = X.Y

     | 1  0  0 |   | C  0 -D |
M' = | 0  A -B | . | 0  1  0 |
     | 0  B  A |   | D  0  C |

     | 1.C + 0.0 +  0.D   1.0 + 0.1 +  0.0   1.-D + 0.0 +  0.C |
M' = | 0.C + A.0 + -B.D   0.0 + A.1 + -B.0   0.-D + A.0 + -B.C |
     | 0.C + B.0 +  A.D   0.0 + B.1 +  A.0   0.-D + B.0 +  A.C |
```

Упрощаем M':

```
     |  C     0  -D   |
M' = | -B.D   A  -B.C |
     |  A.D   B   A.C |
```

Считаем M:

```
M  = M'.Z

     |  C   0  -D  |   | E -F  0 |
M  = | -BD  A  -BC | . | F  E  0 |
     |  AD  B   AC |   | 0  0  1 |

     |  C.E + 0.F +  -D.0    C.-F + 0.E + -D.0     C.0 + 0.0 +  -D.1 |
M  = | -BD.E + A.F + -BC.0  -BD.-F + A.E + -BC.0  -BD.0 + A.0 + -BC.1 |
     |  AD.E + B.F +  AC.0   AD.-F + B.E +  AC.0   AD.0 + 0.0 +  AC.1 |
```

Упрощаем M:

```
     |  CE      -CF      -D  |
M  = | -BDE+AF  -BDF+AE  -BC |
     |  ADE+BF  -ADF+BE   AC |
```

Конечная матрица вращения будет такой:

```
     |  CE      -CF      -D   0 |
M  = | -BDE+AF   BDF+AE  -BC  0 |
```

```
      |  ADE+BF   -ADF+BE   AC   0 |
      |  0         0        0    1 |
```

Сначала считаются значения A,B,C,D,E и F. Также считаются значения BD и AD так как они попадаются чаще одного раза.

Законченый алгоритм будет таким:

```
A      = cos(angle_x);
B      = sin(angle_x);
C      = cos(angle_y);
D      = sin(angle_y);
E      = cos(angle_z);
F      = sin(angle_z);

AD     =   A * D;
BD     =   B * D;

mat[0] =   C * E;
mat[1] =  -C * F;
mat[2] =  -D;
mat[4] = -BD * E + A * F;
mat[5] =  BD * F + A * E;
mat[6] =  -B * C;
mat[8] =  AD * E + B * F;
mat[9] = -AD * F + B * E;
mat[10]=   A * C;

mat[3] =  mat[7] = mat[11] = mat[12] = mat[13] = mat[14] = 0;
mat[15]=  1;
```

При использовании обычных матричных операций, операция заняла бы 128 умножений, 96 сложений и 80 присваиваний.

Оптимизированный алгоритм требует всего 12 умножений, 6 вычитаний и 18 присваиваний.

Как видно при использовании оптимизированного алгоритма, мы получили прирост производительности в 1000%.

Как мне перевести матрицу вращения в углы Эйлера?

Эта операция обратная к прошлому вопросу.
Дана матрица вращения:

```
      |  CE       -CF       -D   0 |
M   = | -BDE+AF    BDF+AE   -BC   0 |
      |  ADE+BF   -ADF+BE    AC   0 |
      |  0         0         0    1 |
```

где A,B косинус и синус угла по оси X,
C,D косинус и синус угла по оси Y,
E,F косинус и синус угла по оси Z,

При использовании Сишной структуры для матрицы 4x4,
индексы элементов будут такими:

```
     |  0  1  2  3 |
M =  |  4  5  6  7 |
     |  8  9 10 11 |
     | 12 13 14 15 |
```

Сравнивая эти таблицы, можно увидеть, что элемент [2] отвечает значению -D или -sin(Y). Значит, вращение по Y можно рассчитать через арксинус. Передав полученное значение в косинус, получим значение C.

Если C не равно 0, то вращение в X и Z, можно получить из третьей колонки и первого ряда соответственно:

```
ось X:  M[6]  = -BC
        M[10] =  AC

ось Z:  M[0]  =  CE
        M[1]  = -CF
```

Углы могут быть получены из деления каждой пары значений на C и получения ответа через арктангенс.

Если C равно 0, то эти вычисления невозможны. В этом случае поворот по Y будет либо -90, либо 90. Так что D будет иметь значения либо 1, либо -1.

В этом случае мы получаем фиксацию оси. Поворот по оси X и Z будет производиться на одной оси. Это можно увидеть из расчета осей поворота.

```
     |  0.E       -0.F        1    0 |
M  = | -B.1.E+AF   B.1.F+AE  -B.0  0 |
     |  A.1.E+BF  -A.1.F+BE   A.0  0 |
     |  0          0          0    1 |
```

Умножение каждого из них дает:

```
     |  0          0          1    0 |
M  = | -BE+AF      BF+AE      0    0 |
     |  AE+BF     -AF+BE      0    0 |
     |  0          0          0    1 |
```

После перестановки:

```
     |  0          0          1    0 |
M  = | -BE+AF      AE+BF      0    0 |
     |  AE+BF      BE-AF      0    0 |
     |  0          0          0    1 |
```

Это можно увидеть из вида матрицы:

```
     |  0   0  -1   0 |
M  = | -V   W   0   0 |
     |  W   V   0   0 |
     |  0   0   0   1 |
```

Где V = BE-AF и W = AE+BF

Эти два значения отвечают за синус и косинус одной оси вращения.

Алгоритм будет таким:

```
angle_y = D = -asin( mat[2]);        /* Считаем ось Y */
C           =  cos( angle_y );
angle_y    *= RADIANS;

if ( fabs( C ) > 0.005 )             /* ось зафиксирована? */
  {
  trx      =  mat[10] / C;           /* Нет, так что находим угол по X */
  try      = -mat[6]  / C;

  angle_x  = atan2( try, trx ) * RADIANS;

  trx      =  mat[0] / C;            /* находим угол по оси Z */
  try      = -mat[1] / C;

  angle_z  = atan2( try, trx ) * RADIANS;
  }
else                                 /* ось все-таки зафиксирована */
  {
  angle_x  = 0;                      /* Устанавливаем угол по оси X на 0 */

  trx      = mat[5];                 /* И считаем ось Z */
  try      = mat[4];

  angle_z  = atan2( try, trx ) * RADIANS;
  }

angle_x = clamp( angle_x, 0, 360 );  /* Приводим углы к диапазону */
angle_y = clamp( angle_y, 0, 360 );
angle_z = clamp( angle_z, 0, 360 );
```

Как мне сгенерировать матрицу вращения для выбранных осей и углов?

Есть лишь один способ создать этот тип матрицы поворота через математику кватернионов.

Смотри вопрос B54.

Как мне сгенерировать матрицу вращения, проецирующую

один вектор на другой?

Когда создаешь анимационные программы, часто необходимо найти матрицу поворота для проецирования направления одного вектора на другой.

Это проблему можно представить в виде двух векторов исходящих из одной точки. То все поле значений вращения сформирует сферу.

В теории, есть бесконечное число осей вращения и углов, которыми можно проецировать один вектор на другой. Все они лежат в плоскости где все точки лежат на одинаковых расстояниях от обоих векторов.

Однако лишь одно решение имеет практический интерес. Это путь, который будет иметь кротчайшее угловое расстояние между векторами.

Вращение оси по этому пути считается через векторное произведение между двумя векторами:

```
V = Vs x Vf
```

Углы вращения считаются через скалярное умножение двух векторов:

```
            -1
Vangle = cos   ( Vs . Vf )
```

Одно из практических применений этого решения проблемы - поиск кратчайшего воздушного пути между двумя городами. В этом случае, каждый город представлен в виде вектора направления в сферических координатах.

Так как Земля круглая, кратчайший путь будет иметь кратчайшее угловое вращение между двумя городами.

Как мне использовать матрицы для перехода из одной системы координат в другую?

Похоже на предыдущую проблему, необходимо привести одну координатную систему к другой.
Однако, вместо того, чтобы пытаться проецировать одну координатную ось к другой,

необходимо совместить все три оси. Координатные системы представлены в виде матриц 3x3 или 4x4.

Проблема в том, чтобы найти матрицу поворота, которая бы отобразила одну матрицу на другую.
Математически это будет так:

```
Mfinal = Mrot . Morig
```

где Mfinal - конечная система координат,
Morig - оригинальная система координат
Mrot - искомая матрица вращения.

Значит, задача состоит в поиске матрицы Mrot. Это можно сделать

через перестановку в уравнении:

```
                -1
Mfinal . Morig   = Mrot

                        -1
Mrot   = Mfinal . Morig
```

Таким образом, нужную матрицу можно вычислить, умножив
обратную матрицу в оригинальной системе на финальную матрицу вращения.

Для проверки, представим, что и оригинальная и конечная матрица - единичные матрицы.
Тогда, матрица поворота должна совпадать с финальной и обратной к ней.

Единожды посчитанная, матрица вращения может быть превращена в кватернион.

Что такое матрица переноса?

Матрица переноса используется для установки объекта в 3D пространстве без его поворота.
Перенос может быть осуществлен с помощью матричного умножения
лишь при использовании 4x4 матриц.

Если перенос описан вектором [X Y Z], то матрица 4x4 будет такой:

```
     | 1  0  0  X |
     |            |
     | 0  1  0  Y |
M =  |            |
     | 0  0  1  Z |
     |            |
     | 0  0  0  1 |
```

Если вектор равен [0 0 0] то вершины останутся на своих местах.

Что такое матрица скалирования?

Матрица скалирования используется для расширения или
сжимания размеров 3D модели.

Если вектор скалирования равен [X Y Z] то матрица будет такой:

```
        | X  0  0  0 |
        |            |
        | 0  Y  0  0 |
    M = |            |
        | 0  0  Z  0 |
        |            |
        | 0  0  0  1 |
```

Если вектор скалирования равен [1 1 1], то в результате будет
единичная матрица и геометрия останется без изменений.

Что такое матрица сдвига?

Матрица сдвига используется для сдвига 3D модели в стороны.
К примеру, "курсивный" текс требует, чтобы каждый символ был сдвинут вправо.

В трех измерениях возможны 6 сторон сдвига:

- сдвигает X по Y
- сдвигает X по Z
- сдвигает Y по X
- сдвигает Y по Z
- сдвигает Z по X
- сдвигает Z по Y

Все 6 направлений сдвига могут быть собраны в одной матрице:

```
        | 1    Syx  Szx  0 |
        |                  |
        | Sxy  1    Szy  0 |
    M = |                  |
        | Sxz  Syz  1    0 |
        |                  |
        | 0    0    0    1 |
        |                  |
```

Где Sij реализует сдвиг I по J
Так что, Sxy сдвигает X по Y

Теоретически, вращение в трех измерениях может быть комбинацией сдвига в 6 направлениях.

Как мне линейно интерполировать две матрицы?

Даны две матрицы, задача найти путь для определения средних точек
в зависимости от переменной t, которая изменяется в рамках от 0.0 до 1.0.

Это можно сделать переводом двух матриц либо в углы Эйлера, либо в сферические углы (кватернионы)
и вектор переноса. В этом случае каждая матрица превращается в пару 3D векторов.

Интерполировать между этими двумя векторами можно стандартной формулой линейной интерполяции:

```
Vr = Va + t .(Vb - Va )
```

где Vr - результат
Va - начальный вектор
Vb - конечный вектор

Это уравнение можно применять и на вектор переноса и на вектор поворота.

Однажды определенный, конечный перенос и поворот потом может быть переведен обратно в среднюю матрицу.

Как мне сделать кубическую интерполяцию между четырьмя матрицами?

Дано четыре матрицы вращения или переноса, задача - найти способ определения средний точек определённых параметром t.

Это можно сделать при помощи кубической интерполяции.

Как и с линейной интерполяцией четыре матрицы переводятся в соответствующие вектора переноса и вращения (опять же, либо углы Эйлера, либо сферические углы)

Каждый из четырех векторов переводится в один вектор G. С помощью математики сплайнов, этот вектор преобразуется в интерполированную матрицу M

Если вектор задан так:

```
    | x1 x2 x3 x4 |
G = | y1 y2 y3 y4 |
    | z1 z2 z3 z4 |
```

Умножение на начальную матрицу:

```
     |  -4.5    9.0  -5.5  1.0  |
Mb = |  13.5  -22.5   9.0  0.0  |
     | -13.5   18.0  -4.5  0.0  |
     |   4.5   -4.5   1.0  0.0  |
```

создаст интерполированную матрицу Mi:

```
Mi = G .Mb
```

Это можно сделать через стандартную матрично-векторное умножение.

Интерполировать можно при помощи параметрической переменной t:

```
R  = Mi . t

                        |t^3|
| xr |   | A B C D |   |t^2|
| yr | = | E F G H | . |t  |
| zr |   | I J K L |   |1  |
```

Результат может быть преобразован обратно в матрицу вращ ения или переноса.

Необходимо заметить, что пути поворота могут оказаться закрученными,
это нормально, так как алгоритм пытается найти путь минимального вращения
между четырьмя векторами.

Из двух методов, сферические углы вращения дают обычно самую
лучшую интерполяцию для вращения.

Как отрисовать матрицу?

Когда используешь графическое окно для 3D анимации, бывает нужно видеть
как матрица вращения влияет на анимацию.

Однако, просто показ матрицы вращения в виде чисел не очень понятен.

Как альтернатива, можно отрисовать численные данные в виде графиков.

Похоже на графический эквалайзер в стерео, матрица вращения можно
представить в виде графика. Каждый элемент матрицы вращения отрисовывается в
виде отдельной колонки в рамке от -1 до +1.

Матрица 3x3 будет выглядеть так:

```
+--+ +--+ +--+
|##| |  | |  |
+--+ +--+ +--+
|  | |  | |  |
+--+ +--+ +--+

+--+ +--+ +--+
|  | |##| |  |
+--+ +--+ +--+
|  | |  | |  |
+--+ +--+ +--+

+--+ +--+ +--+
|  | |  | |##|
+--+ +--+ +--+
|  | |  | |  |
+--+ +--+ +--+
```

В этом случае матрица вращения - единичная матрица,
где каждый элемент главной диагонали равен 1,
остальные равны 0.

Для удобства, отрицательные значения можно красить другим цветом, нежели положительные.

КВАТЕРНИОНЫ

Что такое кватернионы?

Кватернионы - расширение понятия вращения в трех измерениях в четыре. Это позволяет обойти проблему фиксации оси и дает возможность реализовать гладкое и непрерывное вращение.

Их можно представить в виде дополнительного угла вращения к сферическим координатам, иначе говоря, Долгота, Широта и Вращение.

Кватернион описывается четырьмя числами |x y z w|

Они считаются как комбинация трех координатных осей вращения и угла вращения.

Какое место занимают кватернионы в 3D анимации?

Как говорилось до этого, углы Эйлера страдают от так называемой фиксации оси, из-за которой объект перестает вращаться так, как от него этого ждут, в зависимости от последовательности вращений.

Кватернионы - решение этой проблемы. Вместо вращения объекта через серию поворотов, кватернион дает программисту возможность повернуть объект по любой оси на любой угол.

Вращение все равно происходит через матричную математику. Однако, вместо умножения матриц, кватернион представляет оси вращения вместе и результат переводится в нужную матрицу вращения.

Так как вращение описывается в единичным вектором направления, он может быть также вычислен через векторную математику или из

сферических координат, (долгота, широта).

Кватернионы дают еще один плюс - их можно интерполировать. Что дает гладкое и предсказуемое вращение.

Как вычислить сопряжение кватернионов?

Это можно сделать через изменение полярности векторной части кватерниона.

```
--
Qr =  ( Qr.scalar, -Qr.vector )
```

```
quaternion_conjugate( QUAT *qr, QUAT *qa ){
```

```
    qr -> qw =  qa -> qw;
    qr -> qx = -qa -> qx;
    qr -> qy = -qa -> qy;
    qr -> qz = -qa -> qz;
  }
```

Как вычислить обратный кватернион?

Это эквивалентно вычислению сопряжения кватерниона.
Смотри тут <A HREF="Q49">B49. Как вычислить сопряжение кватернионов?</A>

Как вычислить длину кватерниона?

Длина кватерниона рассчитывается через умножение кватерниона на сопряженный:

```
             ------------
            /      --
|Qr| =  \/     Qr.Qr
```

В коде это будет выглядеть так:

```
QFLOAT quaternion_magnitude( QUAT *qa )
  {
  return( sqrt(qa->qw*qa->qw+
               qa->qx*qa->qx+ qa->qy*qa->qy+qa->qz*qa->qz) )
  }
```

Как нормализовать кватернион?

Кватернион может быть нормализован также как вектор. Сначала вычисляют длину кватерниона. Затем обе части кватерниона делятся на это значение.

Единичный кватернион всегда имеет длину 1.0

Как умножить два кватерниона?

Дано два кватерниона Q1 и Q2, задача - найти комбинацию Qr:

```
Qr = Q1.Q2
```

Это можно сделать так:

```
Qr = Q1.Q2 = ( w1.w2 - v1.v2, w1.v2 + w2.v1 + v1 x v2 )
```

где v1 = (x,y,z) кватерниона Q1

w1 = (w) кватерниона Q1
v2 = (x,y,z) кватерниона Q2
w2 = (w) кватерниона Q2

и . и x - стандартное скалярное и векторное произведение.

В коде это будет выглядеть так:

```
quaternion_multiply( QUAT *qr, QUAT *qa, QUAT *qb )
  {
  qr.scalar = v3_dot( &qa->vector, &qb->vector );

  v3_cross(  &va, &qa->vector, &qb->vector );
  v3_scalef( &vb, &qa->vector, &qb->scalar );
  v3_scalef( &vc, &qb->vector, &qa->scaler );
  v3_add(    &va,         &va, &vb );
  v3_add(    &qr->vector, &va, &vc );

  quaternion_normalise( qr );
  }
```

Как мне перевести кватернион в матрицу вращения?

Пусть кватернион задан так:

```
Q = |X Y Z W|
```

То кватернион может быть переведён в матрицу вращения 3x3 так:

```
      |       2     2                                |
      | 1 - 2Y  - 2Z    2XY - 2ZW      2XZ + 2YW     |
      |                                              |
      |                       2     2                |
M =   | 2XY + 2ZW       1 - 2X  - 2Z   2YZ - 2XW     |
      |                                              |
      |                                       2     2 |
      | 2XZ - 2YW       2YZ + 2XW      1 - 2X  - 2Y  |
      |                                              |
```

Если матрица нужна матрица 4x4, то добавляют
еще один нижний ряд и правую колонку.

Матрица создается по следующему правилу

```
xx      = X * X;
xy      = X * Y;
xz      = X * Z;
xw      = X * W;
```

```
yy      = Y * Y;
yz      = Y * Z;
yw      = Y * W;
zz      = Z * Z;
zw      = Z * W;

mat[0]  = 1 - 2 * ( yy + zz );
mat[1]  =     2 * ( xy - zw );
mat[2]  =     2 * ( xz + yw );

mat[4]  =     2 * ( xy + zw );
mat[5]  = 1 - 2 * ( xx + zz );
mat[6]  =     2 * ( yz - xw );

mat[8]  =     2 * ( xz - yw );
mat[9]  =     2 * ( yz + xw );
mat[10] = 1 - 2 * ( xx + yy );

mat[3]  = mat[7] = mat[11 = mat[12] = mat[13] = mat[14] = 0;
mat[15] = 1;
```

Как мне перевести матрицу в кватернион?

Вращение может быть превращено обратно в кватернион
с помощью следующего алгоритма:

Процесс состоит из следующих шагов:

Посчитать проекцию матрицы T из уравнения:

```
          2     2     2
T = 4 - 4x  - 4y  - 4z

           2    2    2
  = 4( 1 -x  - y  - z )

  = mat[0] + mat[5] + mat[10] + 1
```

Если проекция больше 0,
то делаем "мнгновенное" преобразование.

```
S = 0.5 / sqrt(T)
W = 0.25 / S
X = ( mat[9] - mat[6] ) * S
Y = ( mat[2] - mat[8] ) * S
Z = ( mat[4] - mat[1] ) * S
```

Если проекция меньше или равно 0 ищем в главной диагонали

самое большое значение.

В зависимости от этого значения считаем:

Колонка 0:

```
S  = sqrt( 1.0 + mr[0] - mr[5] - mr[10] ) * 2;

Qx = 0.5 / S;
Qy = (mr[1] + mr[4] ) / S;
Qz = (mr[2] + mr[8] ) / S;
Qw = (mr[6] + mr[9] ) / S;
```

Колонка 1:

```
S  = sqrt( 1.0 + mr[5] - mr[0] - mr[10] ) * 2;

Qx = (mr[1] + mr[4] ) / S;
Qy = 0.5 / S;
Qz = (mr[6] + mr[9] ) / S;
Qw = (mr[2] + mr[8] ) / S;
```

Колонка 2:

```
S  = sqrt( 1.0 + mr[10] - mr[0] - mr[5] ) * 2;

Qx = (mr[2] + mr[8] ) / S;
Qy = (mr[6] + mr[9] ) / S;
Qz = 0.5 / S;
Qw = (mr[1] + mr[4] ) / S;
```

Кватернион задан так:

```
Q = | Qx Qy Qz Qw |
```

Как мне преобразовать угол вращения ось в кватернион?

Даны ось и угол вращения, следующий алгоритм используется для создания кватерниона:

```
sin_a = sin( angle / 2 )
cos_a = cos( angle / 2 )

q -> x    = axis -> x * sin_a
q -> y    = axis -> y * sin_a
q -> z    = axis -> z * sin_a
q -> w    = cos_a

quaternion_normalise( q );
```

Не надо нормализовать кватернион, если величины очень малы.

Как мне преобразовать кватернион в ось и угол?

Кватернион может быть преобразован обратно в ось и угол следующим алгоритмом:

```
quaternion_normalise( qr );

cos_angle  = qr -> qw;
angle      = acos( cos_angle ) * 2 * RADIANS;
sin_angle  = sqrt( 1.0 - cos_angle * cos_angle );

if ( fabs( sin_angle ) < 0.0005 )
  sa = 1;

axis -> vx = qr -> qx / sa;
axis -> vy = qr -> qy / sa;
axis -> vz = qr -> qz / sa;
```

Как мне преобразовать сферические углы вращения в кватернион?

Оси вращения сами по себе могут быть заданы в сферических координатах и угле вращения.

Поэтому, кватернион можно считать так:

```
sin_a   = sin( angle / 2 )
cos_a   = cos( angle / 2 )

sin_lat  = sin( latitude )
cos_lat  = cos( latitude )

sin_long = sin( longitude )
cos_long = cos( longitude )

qx       = sin_a * cos_lat * sin_long
qy       = sin_a * sin_lat
qz       = sin_a * sin_lat * cos_long
qw       = cos_a
```

Как перевести кватернион в сферические углы вращения?

Кватернион преобразуется в сферические координаты так:

```
cos_angle = q -> qw;
sin_angle = sqrt( 1.0 - cos_angle * cos_angle );
angle     = acos( cos_angle ) * 2 * RADIANS;

if ( fabs( sin_angle ) < 0.0005 )
  sa = 1;

tx = q -> qx / sa;
ty = q -> qy / sa;
tz = q -> qz / sa;

latitude = -asin( ty );

if ( tx * tx + tz * tz < 0.0005 )
  longitude  = 0;
else
   longitude = atan2( tx, tz ) * RADIANS;

if( longitude < 0 )
  longitude+= 360.0;
```

Как преобразовать углы Эйлера в кватернион?

Преобразование углов Эйлера в кватернион достигается через умножение кватернионов. Каждый угол вращения преобразуется в пару ось-угол,

с осью отвечающей Эвклидовой оси. Пары ось-угол преобразуются в кватернион и перемножаются. В результате будет нужный кватернион.

Код демонстрирующий это:

```
quaternion_from_euler( QUATERNION *q, VFLOAT ax, VFLOAT ay, VFLOAT az ){
  VECTOR3 vx = { 1, 0, 0 }, vy = { 0, 1, 0 }, vz = { 0, 0, 1 };
  QUATERNION qx, qy, qz, qt;

  quaternion_from_axisangle( qx, &vx, rx );
quaternion_from_axisangle( qy, &vy, ry );
quaternion_from_axisangle( qz, &vz, rz );
  quaternion_multiply( &qt, &qx, &qy );
  quaternion_multiply( &q,  &qt, &qz );
}
```

Как мне с помощью кватернионов линейно интерполировать между матрицами?

Для многих анимационных программ, необходимо интерполировать между

двумя вращениями объекта. Эти позиции могутбыть заданы ключевыми кадрами анимации, или инверсной кинематикой.
Используя этот метод, необходимо знать обе матрицы и задачей является интерполяция между ними. Две матрицы представляют собой начальную и конечную матрицы (MS и MF).

При использовании линейной интерполяции, интерполированные матрицы вращения создаются при помощи уравнения смешивания с параметром T, который меняется от 0.0 до 1.0.

При T=0, интерполированная матрица равна начальной.
При T=1, интерполированная матрица равна конечной.

Интерполяция описывается так:

```
MI = F( MS, MI, T )
```

где F функция смешивания.

На первом этапе определяют матрицу, которая преобразует MS в MF:

```
       -1
 T = Ms   . Mf
```

где Ms - начальная матрица,
Mf - конечная матрица,
и T - матрица преобразования.

Потом эту матрицу преобразуют в угол ось и угол поворота.
Что делается через преобразование матрицы в кватернион,
а затем в ось и угол вращения.

Чтобы создать интерполированную матрицу, необходимо только скалировать угол поворота и преобразовать этот угол и ось обратно в матрицу поворота.

Для матриц 4x4 код будет выглядеть так:

```
m4_transpose(    mt, ms );                /* Обращение            */
m4_mult(         ms, mt, mb );            /* Матрица вращения     */
m4_to_axisangle( ms, axis, angle );       /* ось и угол вращения  */

for( t = 0; t < 1.0; t += 0.05 ){
  m4_from_axisangle( mi, axis, angle * t ); /* Интерполяция */

  ... делаем что-то ...

  }
```

где t фактор интерполяции от 0.0 до 1.0

Как мне с помощью кватернионов сделать кубическую

интерполяцию между матрицами?

Иногда, не хватает или невозможно использовать линейную интерполяцию.
В этом случае используют кубическую.
Чтобы сделать кубическую интерполяцию, нужно четыре матрицы вращения.
Каждая из них переводится в сферические вращения - кватернионы
либо сферические углы вращения.

Их умножают с начальной матрицей для Кардиального сплайна.
Это интерполированная матрица затем используется для определения
сферических углов вращения.

Как только интерполированные координаты известны, интерполированная
матрица вращения может быть создана через перевод в кватернион.

Для матриц 4x4, алгоритм будет таким:

```
for ( n = 0; n < 4; n++ )
  m4_to_spherical( mat[n], &v_sph[n] );      /* Сферические координаты */
  m4_multspline( m_cardinal, v_sph, v_interp ); /* Вектор интерполяции */

...

v3_cubic( v_pos, v_interp, t );            /* Интерполяция */
m4_from_spherical( m_rot, v_pos );        /* Возврат к матрице */
```

Это ЧАВО составлено этим человеком и переведено на русский мной.
Любые предложения и дополнения приветствуются. Шлите их на мыло этому чуваку.

Самая свежая версия статьи может быть найдена тут:
ftp://ftp.netcom.com/pub/he/hexapod/index.html
http://www.glue.umd.edu/~rsrodger

Пожертвования